

Résolution numérique d'équations non-linéaires

Dans ce chapitre nous allons appliquer toutes les notions précédentes sur les suites et les fonctions, à la recherche des **zéros** des fonctions. Plus précisément, nous allons voir quelques méthodes afin de trouver des **approximations** des solutions d'une équation du type $(f(x) = 0)$.

1. La dichotomie

1.1. Principe de la dichotomie

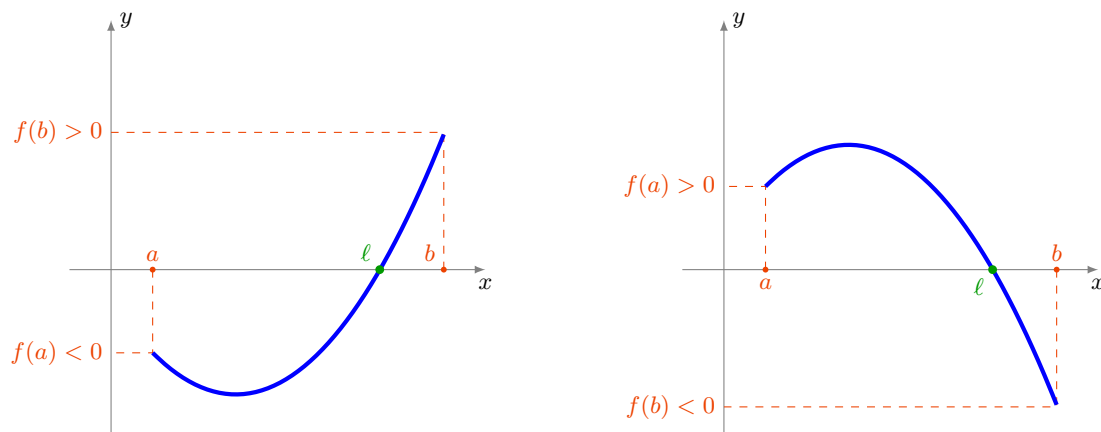
Le principe de dichotomie repose sur la version suivante du **théorème des valeurs intermédiaires** :

Théorème 1.

Soit $f : [a, b] \rightarrow \mathbb{R}$ une fonction **continue** sur un segment.

Si $f(a) \cdot f(b) \leq 0$, alors il existe $\ell \in [a, b]$ tel que $f(\ell) = 0$.

La condition $f(a) \cdot f(b) \leq 0$ signifie que $f(a)$ et $f(b)$ sont de signes opposés (ou que l'un des deux est nul). L'hypothèse de continuité est essentielle !



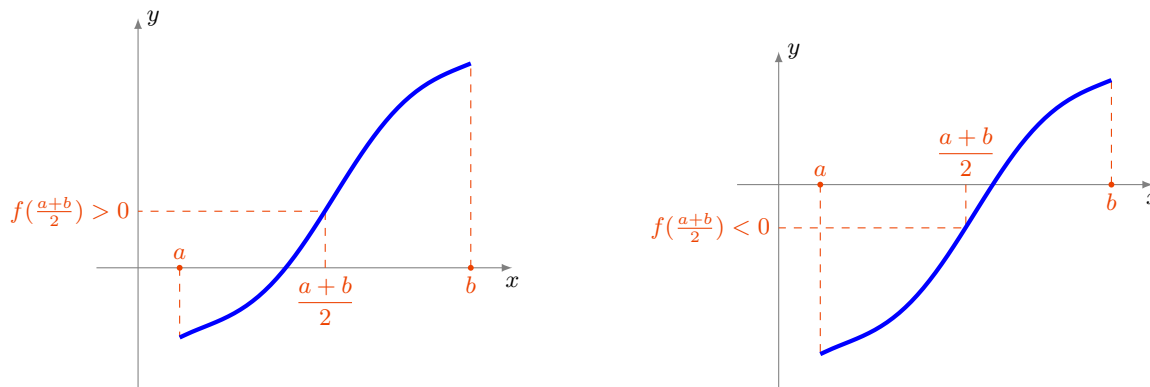
Ce théorème affirme qu'il existe au moins une solution de l'équation $(f(x) = 0)$ dans l'intervalle $[a, b]$. Pour le rendre effectif, et trouver une solution **approchée** de l'équation $(f(x) = 0)$, il s'agit maintenant de l'appliquer sur un intervalle suffisamment petit. On va voir que cela permet d'obtenir un ℓ solution de l'équation $(f(x) = 0)$ comme la limite d'une suite.

Voici comment construire une suite d'intervalles **emboîtés**, dont la longueur tend vers 0, et contenant chacun une solution de l'équation $(f(x) = 0)$.

On part d'une fonction $f : [a, b] \rightarrow \mathbb{R}$ continue, avec $a < b$, et $f(a) \cdot f(b) \leq 0$.

Voici la première étape de la construction : on regarde le signe de la valeur de la fonction f appliquée au point milieu $\frac{a+b}{2}$.

- Si $f(a) \cdot f(\frac{a+b}{2}) \leq 0$, alors il existe $c \in [a, \frac{a+b}{2}]$ tel que $f(c) = 0$.
- Si $f(a) \cdot f(\frac{a+b}{2}) > 0$, cela implique que $f(\frac{a+b}{2}) \cdot f(b) \leq 0$, et alors il existe $c \in [\frac{a+b}{2}, b]$ tel que $f(c) = 0$.



Nous avons obtenu un intervalle de longueur moitié dans lequel l'équation $(f(x) = 0)$ admet une solution. On itère alors le procédé pour diviser de nouveau l'intervalle en deux.

Voici le processus complet :

- **Au rang 0 :**

On pose $a_0 = a$, $b_0 = b$. Il existe une solution x_0 de l'équation $(f(x) = 0)$ dans l'intervalle $[a_0, b_0]$.

- **Au rang 1 :**

- Si $f(a_0) \cdot f(\frac{a_0+b_0}{2}) \leq 0$, alors on pose $a_1 = a_0$ et $b_1 = \frac{a_0+b_0}{2}$,
- sinon on pose $a_1 = \frac{a_0+b_0}{2}$ et $b_1 = b_0$.
- Dans les deux cas, il existe une solution x_1 de l'équation $(f(x) = 0)$ dans l'intervalle $[a_1, b_1]$.

- ...

- **Au rang n :** supposons construit un intervalle $[a_n, b_n]$, de longueur $\frac{b-a}{2^n}$, et contenant une solution x_n de l'équation $(f(x) = 0)$. Alors :

- Si $f(a_n) \cdot f(\frac{a_n+b_n}{2}) \leq 0$, alors on pose $a_{n+1} = a_n$ et $b_{n+1} = \frac{a_n+b_n}{2}$,
- sinon on pose $a_{n+1} = \frac{a_n+b_n}{2}$ et $b_{n+1} = b_n$.
- Dans les deux cas, il existe une solution x_{n+1} de l'équation $(f(x) = 0)$ dans l'intervalle $[a_{n+1}, b_{n+1}]$.

À chaque étape on a

$$a_n \leq x_n \leq b_n.$$

On arrête le processus dès que $b_n - a_n = \frac{b-a}{2^n}$ est inférieur à la précision souhaitée.

Comme (a_n) est par construction une suite croissante, (b_n) une suite décroissante, et $(b_n - a_n) \rightarrow 0$ lorsque $n \rightarrow +\infty$, les suites (a_n) et (b_n) sont adjacentes et donc elles admettent une même limite. D'après le théorème des gendarmes, c'est aussi la limite disons ℓ de la suite (x_n) . La continuité de f montre que $f(\ell) = \lim_{n \rightarrow +\infty} f(x_n) = \lim_{n \rightarrow +\infty} 0 = 0$. Donc les suites (a_n) et (b_n) tendent toutes les deux vers ℓ , qui est une solution de l'équation $(f(x) = 0)$.

1.2. Calcul de l'erreur

Il est donc facile d'avoir une majoration de l'erreur. En effet, à chaque étape, la taille l'intervalle contenant ℓ est divisée par 2. Au départ, on sait que $\ell \in [a, b]$ (de longueur $b - a$) ; puis $\ell \in [a_1, b_1]$ (de longueur $\frac{b-a}{2}$) ; puis $\ell \in [a_2, b_2]$ (de longueur $\frac{b-a}{4}$) ; ... ; $[a_n, b_n]$ étant de longueur $\frac{b-a}{2^n}$.

Si, par exemple, on souhaite obtenir une approximation de ℓ à 10^{-N} près, comme on sait que $a_n \leq \ell \leq b_n$, on obtient $|\ell - a_n| \leq |b_n - a_n| = \frac{b-a}{2^n}$. Donc pour avoir $|\ell - a_n| \leq 10^{-N}$, il suffit de choisir n tel que $\frac{b-a}{2^n} \leq 10^{-N}$.

Nous allons utiliser le logarithme décimal :

$$\frac{b-a}{2^n} \leq 10^{-N} \iff n \geq \frac{N + \log(b-a)}{\log 2}$$

Sachant $\log 2 = 0,301\dots$, si par exemple $b-a \leq 1$, voici le nombre d'itérations suffisantes pour avoir une précision de 10^{-N} (ce qui correspond, à peu près, à N chiffres exacts après la virgule).

10^{-10} (~ 10 décimales)	34 itérations
10^{-100} (~ 100 décimales)	333 itérations
10^{-1000} (~ 1000 décimales)	3322 itérations

Il faut entre 3 et 4 itérations supplémentaires pour obtenir une nouvelle décimale.

1.3. Algorithmes

Voici comment implémenter la dichotomie dans le langage **Python**. Tout d'abord on définit une fonction f (ici par exemple $f(x) = x^2 - 10$) :

Code 1 (*dichotomie.py* (1)).

```
def f(x):
    return x*x - 10
```

Puis la dichotomie proprement dite : en entrée de la fonction, on a pour variables a, b et n le nombre d'étapes voulues.

Code 2 (*dichotomie.py* (2)).

```
def dichotomie(a,b,n):
    for i in range(n):
        c = (a+b)/2
        if f(a)*f(c) <= 0:
            b = c
        else:
            a = c
    return a,b
```

Même algorithme, mais avec cette fois en entrée la précision souhaitée :

Code 3 (*dichotomie.py* (3)).

```
def dichobis(a,b,prec):
    while b-a > prec:
        c = (a+b)/2
        if f(a)*f(c) <= 0:
```

```

        b = c
    else:
        a = c
    return a,b

```

Enfin, voici la version récursive de l'algorithme de dichotomie.

Code 4 (*dichotomie.py (4)*).

```

def dichotomie(a,b,prec):
    if b-a<=prec:
        return a,b
    else:
        c = (a+b)/2
        if f(a)*f(c) <= 0:
            return dichotomie(a,c,prec)
        else:
            return dichotomie(c,b,prec)

```

Travaux pratiques 1.

1. Tracer le graphe de la fonction $f(x) = 5x^2 + 2x - 34$.
2. Écrire la fonction `dichotomie`, prenant en entrée $f, a, b, \text{epsilon}$, et qui tiendra compte d'une possible erreur dans le déroulement du programme. Tester le programme avec f ci-dessus.

Code 5 (*dicho-et-exception.py*).

```

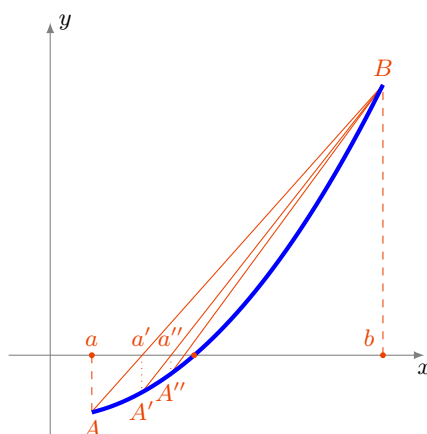
def dichotomie(f,a,b,epsilon):
    try:
        if f(a)*f(b) > 0:    # On vérifie l'encadrement de la fonction
            raise Exception("Mauvais choix de a ou b")
        else:
            m = (a+b)/2.
            while abs(a - b) > epsilon:
                if f(m) == 0.:
                    return m
                elif f(a)*f(b) > 0:
                    a = m
                else:
                    b = m
                m = (a+b)/2.
            return m
    except:
        print("Mauvais choix de l'encadrement")
        return 0

```

2. La méthode de la sécante

2.1. Principe de la sécante

L'idée de la méthode de la sécante est très simple : pour une fonction f continue sur un intervalle $[a, b]$, et vérifiant $f(a) \leq 0$, $f(b) > 0$, on trace le segment $[AB]$ où $A = (a, f(a))$ et $B = (b, f(b))$. Si le segment reste au-dessus du graphe de f alors la fonction s'annule sur l'intervalle $[a', b]$ où $(a', 0)$ est le point d'intersection de la droite (AB) avec l'axe des abscisses. La droite (AB) s'appelle la **sécante**. On recommence en partant maintenant de l'intervalle $[a', b]$ pour obtenir une valeur a'' .

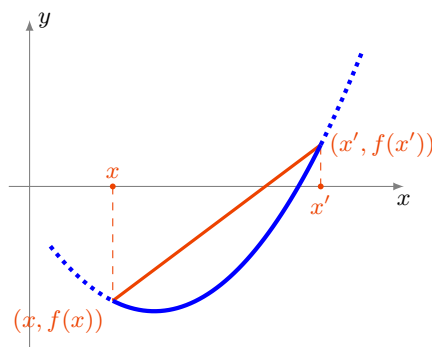

Proposition 1.

Soit $f : [a, b] \rightarrow \mathbb{R}$ une fonction continue, strictement croissante et convexe telle que $f(a) \leq 0$, $f(b) > 0$. Alors la suite définie par

$$a_0 = a \quad \text{et} \quad a_{n+1} = a_n - \frac{b - a_n}{f(b) - f(a_n)} f(a_n)$$

est croissante et converge vers la solution ℓ de $(f(x) = 0)$.

L'hypothèse f **convexe** signifie exactement que pour tout x, x' dans $[a, b]$ la sécante (ou corde) entre $(x, f(x))$ et $(x', f(x'))$ est au-dessus du graphe de f .



2.2. Calcul de l'erreur

La méthode de la sécante fournit l'encadrement $a_n \leq \ell \leq b$. Mais comme b est fixe cela ne donne pas d'information exploitable pour $|\ell - a_n|$. Voici une façon générale d'estimer l'erreur, à l'aide du théorème des accroissements finis.

Proposition 2.

Soit $f : I \rightarrow \mathbb{R}$ une fonction dérivable et ℓ tel que $f(\ell) = 0$. S'il existe une constante $m > 0$ telle que

pour tout $x \in I$, $|f'(x)| \geq m$ alors

$$|x - \ell| \leq \frac{|f(x)|}{m} \quad \text{pour tout } x \in I.$$

Démonstration. Par l'inégalité des accroissements finis entre x et ℓ : $|f(x) - f(\ell)| \geq m|x - \ell|$ mais $f(\ell) = 0$, d'où la majoration. $\square$

Exemple 1 (Erreur pour $\sqrt{10}$).

Soit $f(x) = x^2 - 10$ et l'intervalle $I = [3, 4]$. Alors $f'(x) = 2x$ donc $|f'(x)| \geq 6$ sur I . On pose donc $m = 6$, $\ell = \sqrt{10}$, $x = a_n$. On obtient l'estimation de l'erreur :

$$\epsilon_n = |\ell - a_n| \leq \frac{|f(a_n)|}{m} = \frac{|a_n^2 - 10|}{6}$$

Par exemple on a trouvé $a_2 = 3,16\dots \leq 3,17$ donc $\sqrt{10} - a_2 \leq \frac{|3,17^2 - 10|}{6} = 0,489$.

Pour a_8 on a trouvé $a_8 = 3,1622776543347473\dots$ donc $\sqrt{10} - a_8 \leq \frac{|a_8^2 - 10|}{6} = 6,14\dots \cdot 10^{-9}$. On a en fait 7 décimales exactes après la virgule.

Dans la pratique, voici le nombre d'itérations suffisantes pour avoir une précision de 10^{-n} pour cet exemple. Grosso-modo, une itération de plus donne une décimale supplémentaire.

10^{-10} (~ 10 décimales)	10 itérations
10^{-100} (~ 100 décimales)	107 itérations
10^{-1000} (~ 1000 décimales)	1073 itérations

2.3. Algorithme

Voici l'algorithme : c'est tout simplement la mise en œuvre de la suite récurrente (a_n) .

Code 6 (*secante.py*).

```
def secante(a,b,n):
    for i in range(n):
        a = a-f(a)*(b-a)/(f(b)-f(a))
    return a
```

Mini-exercices.

1. À la main, calculer un encadrement à 0,1 près de $\sqrt{3}$. Idem avec $\sqrt[3]{2}$.
2. Calculer une approximation des solutions de l'équation $x^3 + 1 = 3x$.
3. Calculer une approximation de la solution de l'équation $(\cos x = 0)$ sur $[0, \pi]$. Idem avec $(\cos x = 2 \sin x)$.
4. Étudier l'équation $(\exp(-x) = -\ln(x))$. Donner une approximation de la (ou des) solution(s) et une majoration de l'erreur correspondante.

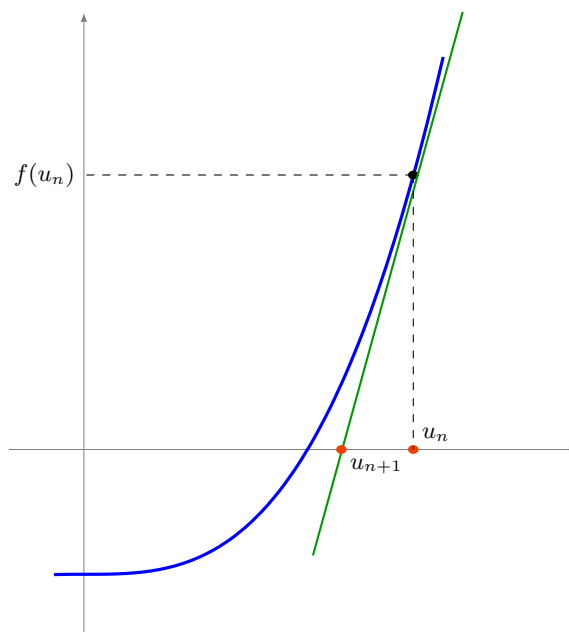
3. La méthode de Newton

3.1. Méthode de Newton

La méthode de Newton consiste à remplacer la sécante de la méthode précédente par la tangente. Elle est d'une redoutable efficacité.

Partons d'une fonction dérivable $f : [a, b] \rightarrow \mathbb{R}$ et d'un point $u_0 \in [a, b]$. On appelle $(u_1, 0)$ l'intersection de la tangente au graphe de f en $(u_0, f(u_0))$ avec l'axe des abscisses. Si $u_1 \in [a, b]$ alors on recommence l'opération avec la tangente au point d'abscisse u_1 . Ce processus conduit à la définition d'une suite récurrente :

$$u_0 \in [a, b] \quad \text{et} \quad u_{n+1} = u_n - \frac{f(u_n)}{f'(u_n)}.$$



3.2. Résultats pour $\sqrt{10}$

Pour calculer $\sqrt{a}$, on pose $f(x) = x^2 - a$, avec $f'(x) = 2x$. La suite issue de la méthode de Newton est déterminée par $u_0 > 0$ et la relation de récurrence $u_{n+1} = u_n - \frac{u_n^2 - a}{2u_n}$. Suite qui pour cet exemple s'appelle **suite de Héron** et que l'on récrit souvent

$$u_0 > 0 \quad \text{et} \quad u_{n+1} = \frac{1}{2} \left(u_n + \frac{a}{u_n} \right).$$

Proposition 3.

Cette suite (u_n) converge vers $\sqrt{a}$.

Pour le calcul de $\sqrt{10}$, on pose par exemple $u_0 = 4$, et on peut même commencer les calculs à la main :

$$\begin{aligned}
u_0 &= 4 \\
u_1 &= \frac{1}{2} \left(u_0 + \frac{10}{u_0} \right) = \frac{1}{2} \left(4 + \frac{10}{4} \right) = \frac{13}{4} = 3,25 \\
u_2 &= \frac{1}{2} \left(u_1 + \frac{10}{u_1} \right) = \frac{1}{2} \left(\frac{13}{4} + \frac{10}{\frac{13}{4}} \right) = \frac{329}{104} = 3,1634\dots \\
u_3 &= \frac{1}{2} \left(u_2 + \frac{10}{u_2} \right) = \frac{216401}{68432} = 3,16227788\dots \\
u_4 &= 3,162277660168387\dots
\end{aligned}$$

Pour u_4 on obtient $\sqrt{10} = 3,1622776601683\dots$ avec déjà 13 décimales exactes !

3.3. Calcul de l'erreur pour $\sqrt{10}$

Proposition 4.

1. Soit k tel que $u_1 - \sqrt{a} \leq k$. Alors pour tout $n \geq 1$:

$$u_n - \sqrt{a} \leq 2\sqrt{a} \left(\frac{k}{2\sqrt{a}} \right)^{2^{n-1}}$$

2. Pour $a = 10$, $u_0 = 4$, on a :

$$u_n - \sqrt{10} \leq 8 \left(\frac{1}{24} \right)^{2^{n-1}}$$

Démonstration.

1. À partir de la relation de récurrence $u_{n+1} = u_n - \frac{u_n^2 - a}{2u_n}$, nous avons l'égalité :

$$u_{n+1}^2 - a = \frac{(u_n^2 - a)^2}{4u_n^2} \text{ donc } (u_{n+1} - \sqrt{a})(u_{n+1} + \sqrt{a}) = \frac{(u_n - \sqrt{a})^2(u_n + \sqrt{a})^2}{4u_n^2}$$

Ainsi comme $u_n \geq \sqrt{a}$ pour $n \geq 1$:

$$u_{n+1} - \sqrt{a} = (u_n - \sqrt{a})^2 \times \frac{1}{u_{n+1} + \sqrt{a}} \times \frac{1}{4} \left(1 + \frac{\sqrt{a}}{u_n} \right)^2 \leq (u_n - \sqrt{a})^2 \times \frac{1}{2\sqrt{a}} \times \frac{1}{4} \cdot (1+1)^2 = \frac{1}{2\sqrt{a}} (u_n - \sqrt{a})^2$$

Si k vérifie $u_1 - \sqrt{a} \leq k$, nous allons en déduire par récurrence, pour tout $n \geq 1$, la formule

$$u_n - \sqrt{a} \leq 2\sqrt{a} \left(\frac{k}{2\sqrt{a}} \right)^{2^{n-1}}$$

C'est vrai pour $n = 1$. Supposons la formule vraie au rang n , alors :

$$u_{n+1} - \sqrt{a} \leq \frac{1}{2\sqrt{a}} (u_n - \sqrt{a})^2 = \frac{1}{2\sqrt{a}} (2\sqrt{a})^2 \left(\left(\frac{k}{2\sqrt{a}} \right)^{2^{n-1}} \right)^2 = 2\sqrt{a} \left(\frac{k}{2\sqrt{a}} \right)^{2^n}$$

La formule est donc vraie au rang suivant.

2. Pour $a = 10$ avec $u_0 = 4$ on a $u_1 = 3,25$. Comme $3 \leq \sqrt{10} \leq 4$ alors $u_1 - \sqrt{10} \leq u_1 - 3 \leq \frac{1}{4}$. On fixe donc $k = \frac{1}{4}$. Toujours par l'encadrement $3 \leq \sqrt{10} \leq 4$, la formule obtenue précédemment devient

$$u_n - \sqrt{a} \leq 2 \cdot 4 \left(\frac{\frac{1}{4}}{2 \cdot 3} \right)^{2^{n-1}} = 8 \left(\frac{1}{24} \right)^{2^{n-1}}.$$

□

Admirez la puissance de la méthode de Newton : 11 itérations donnent déjà 1000 décimales exactes après la virgule. Cette rapidité de convergence se justifie grâce au calcul de l'erreur : la précision est multipliée par 2 à chaque étape, donc à chaque itération le nombre de décimales exactes double !

10^{-10} (~ 10 décimales)	4 itérations
10^{-100} (~ 100 décimales)	8 itérations
10^{-1000} (~ 1000 décimales)	11 itérations

3.4. Algorithme

Voici l'algorithme pour le calcul de $\sqrt{a}$. On précise en entrée le réel $a \geq 0$ dont on veut calculer la racine et le nombre n d'itérations.

Code 7 (*newton.py*).

```
def racine_carree(a,n):
    u=4 # N'importe qu'elle valeur > 0
    for i in range(n):
        u = 0.5*(u+a/u)
    return u
```

En utilisant le module `decimal` le calcul de u_n pour $n = 11$ donne 1000 décimales de $\sqrt{10}$:

3,

```
16227766016837933199889354443271853371955513932521 68268575048527925944386392382213442481083793002951
87347284152840055148548856030453880014690519596700 15390334492165717925994065915015347411333948412408
53169295770904715764610443692578790620378086099418 28371711548406328552999118596824564203326961604691
31433612894979189026652954361267617878135006138818 62785804636831349524780311437693346719738195131856
78403231241795402218308045872844614600253577579702 82864402902440797789603454398916334922265261206779
26516760310484366977937569261557205003698949094694 21850007358348844643882731109289109042348054235653
40390727401978654372593964172600130699000095578446 31096267906944183361301813028945417033158077316263
86395193793704654765220632063686587197822049312426 05345411160935697982813245229700079888352375958532
85792513629646865114976752171234595592380393756251 25369855194955325099947038843990336466165470647234
99979613234340302185705218783667634578951073298287 51579452157716521396263244383990184845609357626020
```

Mini-exercices.

1. À la calculatrice, calculer les trois premières étapes pour une approximation de $\sqrt{3}$, sous forme de nombres rationnels. Idem avec $\sqrt[3]{2}$.
2. Implémenter la méthode de Newton, étant données une fonction f et sa dérivée f' .
3. Calculer une approximation des solutions de l'équation $x^3 + 1 = 3x$.
4. Soit $a > 0$. Comment calculer $\frac{1}{a}$ par une méthode de Newton ?
5. Calculer n de sorte que $u_n - \sqrt{10} \leq 10^{-\ell}$ (avec $u_0 = 4$, $u_{n+1} = \frac{1}{2} \left(u_n + \frac{a}{u_n} \right)$, $a = 10$).

4. Autres méthodes

4.1. Méthode de régula-falsi

Cette méthode est une variante de la **Dichotomie** : au lieu de choisir, pour découper l'intervalle $[a_n, b_n]$, le milieu c_n , on considère l'intersection de la corde joignant les points $A_n(a_n, f(a_n))$ et $B_n(b_n, f(b_n))$ avec l'axe des abscisses. On pose alors

$$c_n = a_n - f(a_n) \frac{b_n - a_n}{f(b_n) - f(a_n)}$$

La convergence est souvent plus rapide qu'avec la méthode de la dichotomie. Pour écrire un fonction **RegulaFalsi**, il suffit de modifier la fonction **Dichotomie** (c'est un exemple de polymorphisme).

4.2. Méthode du point fixe

On se propose de résoudre une équation de la forme

$$f(x) = x.$$

Les solutions de cette équation sont appelés des **points fixes** de f ; géométriquement, ce sont les abscisses des points d'intersection entre la courbe représentative de f et la première bissectrice des axes. Une méthode fructueuse pour résoudre ce type d'équation est de procéder par itération, à condition toutefois que soient vérifiées par exemple les hypothèses du théorème du point fixe, dont voici l'énoncé :

Proposition 5.

Soit J un intervalle **fermé stable** pour une application **contractante** f de **rapport de Lipschitz** $k \in [0, 1[$; alors

- f possède un unique point fixe $l \in J$;
- pour tout $\alpha \in J$, la suite $\begin{cases} x_0 = \alpha \\ \forall n \in \mathbb{N} \quad x_{n+1} = f(x_n) \end{cases}$ converge vers l et on a la majoration $\forall n \in \mathbb{N}, \quad |x_n - l| \leq k^n |x_0 - l|$

Définition 1.

On dit qu'une méthode des points fixes converge à l'ordre p si :

$$|e_{n+1}| \approx C |e_n|^p$$

où C est une constante et e_n l'erreur à l'étape n .

La convergence d'ordre 1 est également dite **linéaire**, tandis que celle d'ordre 2 est dite **quadratique**.

Remarque.

Le **taux de convergence** d'une méthode des points fixes est donné par $|f'(l)|$.

Les points fixes peuvent être classés en différentes catégories suivant la valeur de $f'(l)$ en points fixes **attractifs** ($|f'(l)| < 1$), **répulsifs** ($|f'(l)| > 1$), **super-attractifs** ($|f'(l)| = 0$), **indifférents** ($|f'(l)| = 1$).

4.3. Comparatif des différentes méthodes

Comparons les différentes méthodes de résolution sur la recherche d'une **approximation décimale** de $\sqrt{2}$.

D'une part, on peut appliquer les méthodes précédentes (hormis celle du point fixe) à l'équation $x^2 - 2 = 0$. D'autre part, on peut appliquer la méthode du point fixe en réécrivant cette équation sous la $x = f(x)$ avec $f(x) = \frac{1}{2}(x + \frac{a}{x})$.

Mini-exercices.

Soit

$$f(x) = \frac{1}{2}(x + \frac{a}{x})$$

Prouver que f est stable sur $]\sqrt{a}, +\infty[$; y est $\frac{1}{2}$ -lipschitzienne et possède sur cet intervalle pour unique point fixe le réel $\sqrt{a}$, qui est un point super-attractif.

Travaux pratiques 2.

1. Écrire un programme qui réalise la comparaison en affichant l'erreur absolue (ci-dessous).
2. Expliquer pourquoi La méthode du point fixe converge ici à une vitesse vertigineuse ?
3. Vérifier que même si la méthode de Newton permet en général d'obtenir une convergence quadratique, un mauvais choix de la valeur initiale peut provoquer la divergence.

MaxIter	Dichotomie	RegulaFalsi	Secante	PointFixe	Newton
1	8.579e-02	8.0880e-02	8.088e-02	2.453e-03	2.453e-03
2	1.642e-01	1.4358e-02	1.421e-02	2.124e-06	2.124e-06
3	2.892e-01	4.2046e-04	4.206e-04	1.595e-12	1.595e-12
4	2.267e-01	4.2046e-04	2.124e-06	2.220e-16	0.000e+00
5	1.955e-01	4.2046e-04	3.158e-10	2.220e-16	2.220e-16
6	1.798e-01	4.2046e-04	2.220e-16	2.220e-16	2.220e-16
7	1.720e-01	4.2046e-04	0.000e+00	2.220e-16	2.220e-16
8	1.681e-01	4.2046e-04	2.220e-16	2.220e-16	2.220e-16